

open the video but it is not otherwise viewable. This might be what you want in your CMS.

By supplying a filtered queryset to the `ArticleDetail` view we limit the articles it is allowed to show to the ones that are filtered. Any article not in the list of filtered articles might as well not exist as far as the view is concerned. Again, neither approach wrong, both have their uses.

Since this code is very simple, we won't bother discussing all the different views we need to have. Rather we will just discuss some of the more interesting cases that test the limits of generic views.

The first view that we will have trouble implementing with the generic views we have discussed so far is the home page itself. We would want this page to list a few different types of content, and generic views only support pulling details from a single model per view. For this page we could either use a custom class-based view, or a function-based view.

The other view that might cause trouble with these views is the tag list page. We would like to render this as a tag cloud, and in order to do that we need extra details, such as the number of times each tag is used, and this isn't something that is available by default when listing tags.

The author detail view isn't special at all, except that we want to be able to link to authors by their username rather than a numeric ID or slug (since users don't have a slug). A user's username can be used as a slug, but this won't happen automatically, we will need to configure the `ArticleDetail` class to perform this conversion.

Finally, our gallery detail view which will need to support multiple layouts. While this is an interesting case, it is entirely possible to do with a standard `DetailView` by using some basic logic in the template rather than in the view.

Since these are the troubling cases, we'll look at these in more detail.

The Author Detail View

Let's start with the simplest of all cases, the `AuthorDetail` view which is just a matter of configuring the view to look for the slug information in the right place.

```
• class AuthorDetail(DetailView):
•     model = User
•
•     def get_slug_field(self):
•         return 'username'
```